

WEEK 5

NAME : P. Varsha

VTU : VTU29196

COURSE NAME : Problem solving and testing using java

COURSE CODE :10211CS227

TASKS:

1.MAXIMUM SUBARRAY

2.SUBARRAY DIVISION

3 MAX SUBARRAY

4.MAXIMUM SUM CIRCULAR SUBARRAY

5.STRING TO INTEGER

6.ALTERNATING CHARACTERS

7.LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS

8.FIND AND REPLACE PATTERN

9.STRING MATCHING IN AN ARRAY

10.NAIVE ALGORITHM FOR PATTERN SEARCHING

1. Given an integer array `nums`, find the subarray with the largest sum, and return its sum.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

Output: 6

Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`

Output: 1

Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`

Output: 23

Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

$1 \leq \text{nums.length} \leq 105$

$-104 \leq \text{nums}[i] \leq 104$

Program:

```
class Solution {
```

```

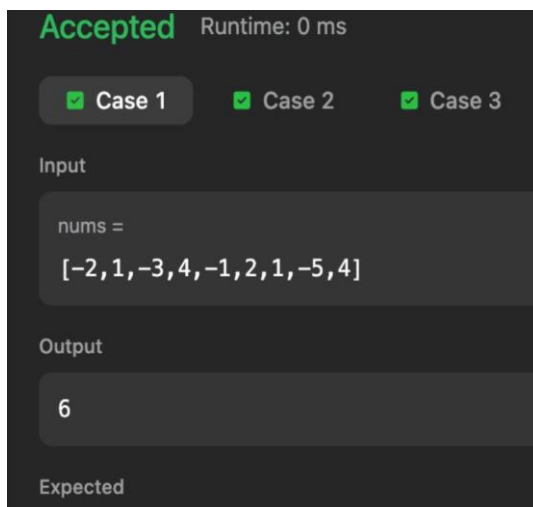
public int maxSubArray(int[] nums) {
    int currentSum = nums[0];
    int maxSum = nums[0];

    for (int i = 1; i < nums.length; i++) {
        currentSum = Math.max(nums[i], currentSum + nums[i]);
        maxSum = Math.max(maxSum, currentSum);
    }

    return maxSum;
}

```

Output:



2. Two children, Lily and Ron, want to share a chocolate bar. Each of the squares has an integer on it.

Lily decides to share a contiguous segment of the bar selected such that:

The length of the segment matches Ron's birth month, and,

The sum of the integers on the squares is equal to his birth day.

Determine how many ways she can divide the chocolate.

Example

Lily wants to find segments summing to Ron's birth day, with a length equalling his birth month, . In this case, there are two segments meeting her criteria: and .

Function Description

Complete the birthday function in the editor below.

birthday has the following parameter(s):

int s[n]: the numbers on each of the squares of chocolate

int d: Ron's birth day

int m: Ron's birth month

Returns

int: the number of ways the bar can be divided

Input Format

The first line contains an integer , the number of squares in the chocolate bar.

The second line contains space-separated integers , the numbers on the chocolate squares where .

The third line contains two space-separated integers, and , Ron's birth day and his birth month.

Constraints

, where ()

Sample Input 0

5

1 2 1 3 2

3 2

Sample Output 0

2

Explanation 0

Lily wants to give Ron squares summing to . The following two segments meet the criteria:

image

Sample Input 1

6

1 1 1 1 1 1

3 2

Sample Output 1

0

Explanation 1

Lily only wants to give Ron consecutive squares of chocolate whose integers sum to .
There are no possible pieces satisfying these constraints:

image

Thus, we print as our answer.

Sample Input 2

1

4

4 1

Sample Output 2

1

Program:

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;
```

```
class Result {

    /**
     * Complete the 'birthday' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts following parameters:
     * 1. INTEGER_ARRAY s
     * 2. INTEGER d
     * 3. INTEGER m
     */

    public static int birthday(List<Integer> s, int d, int m) {
        if (s.size() < m) {
            return 0;
        }

        int count = 0;
        int sum = 0;

        // Calculate sum of the first window
        for (int i = 0; i < m; i++) {
            sum += s.get(i);
        }

        if (sum == d) {
            count++;
        }
    }
}
```

```

// Slide the window
for (int i = m; i < s.size(); i++) {
    sum += s.get(i);    // Add new element
    sum -= s.get(i - m); // Remove old element

    if (sum == d) {
        count++;
    }
}

return count;

}

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        int n = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> s = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split("
"))
            .map(Integer::parseInt)

```



```

        .collect(toList());

String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$", "").split("
");

int d = Integer.parseInt(firstMultipleInput[0]);

int m = Integer.parseInt(firstMultipleInput[1]);

int result = Result.birthday(s, d, m);

bufferedWriter.write(String.valueOf(result));
bufferedWriter.newLine();

bufferedReader.close();
bufferedWriter.close();
}
}

```

Output:

Input (stdin)	
1	5
2	1 2 1 3 2
3	3 2
Your Output (stdout)	
1	2
Expected Output	
1	2

3. We define subsequence as any subset of an array. We define a subarray as a contiguous subsequence in an array.

Given an array, find the maximum possible sum among:

all nonempty subarrays.

all nonempty subsequences.

Print the two values as space-separated integers on one line.

Note that empty subarrays/subsequences should not be considered.

Example

The maximum subarray sum is comprised of elements at indices . Their sum is . The maximum subsequence sum is comprised of elements at indices and their sum is .

Function Description

Complete the `maxSubarray` function in the editor below.

`maxSubarray` has the following parameter(s):

`int arr[n]`: an array of integers

Returns

`int[2]`: the maximum subarray and subsequence sums

Input Format

The first line of input contains a single integer , the number of test cases.

The first line of each test case contains a single integer .

The second line contains space-separated integers where .

Constraints

The subarray and subsequences you consider should have at least one element.

Sample Input 0

2

4

1 2 3 4

6

2 -1 2 3 4 -5

Sample Output 0

10 10

10 11

Explanation 0

In the first case: The maximum sum for both types of subsequences is just the sum of all the elements since they are all positive.

In the second case: The subarray is the subarray with the maximum sum, and is the subsequence with the maximum sum.

Sample Input 1

1

5

-2 -3 -1 -4 -6

Sample Output 1

-1 -1

Program:

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    /**
     * Complete the 'maxSubarray' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts INTEGER_ARRAY arr as parameter.
     */

    public static List<Integer> maxSubarray(List<Integer> arr) {
```

```
int currentSum = arr.get(0);
int maxSubarraySum = arr.get(0);

// Maximum Subsequence Sum
int maxSubsequenceSum = 0;
int maxElement = arr.get(0);

for (int i = 0; i < arr.size(); i++) {
    int num = arr.get(i);

    // For subsequence
    if (num > 0) {
        maxSubsequenceSum += num;
    }
    maxElement = Math.max(maxElement, num);

    // Kadane's Algorithm
    if (i > 0) {
        currentSum = Math.max(num, currentSum + num);
        maxSubarraySum = Math.max(maxSubarraySum, currentSum);
    }
}

// If all elements are negative
if (maxSubsequenceSum == 0) {
    maxSubsequenceSum = maxElement;
}
```

```

return Arrays.asList(maxSubarraySum, maxSubsequenceSum);

}

}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        int t = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, t).forEach(tltr -> {

            try {

                int n = Integer.parseInt(bufferedReader.readLine().trim());

                List<Integer> arr = Stream.of(bufferedReader.readLine().replaceAll("\\s+$",
""),).split(" ")

                    .map(Integer::parseInt)

                    .collect(toList());

                List<Integer> result = Result.maxSubarray(arr);

                bufferedWriter.write(

                    result.stream()

                        .map(Object::toString)

```

```

        .collect(joining(" "))
        + "\n"
    );
} catch (IOException ex) {
    throw new RuntimeException(ex);
}
});

bufferedReader.close();
bufferedWriter.close();
}
}

```

Output:

Input (stdin)	
1	2
2	4
3	1 2 3 4
4	6
5	2 -1 2 3 4 -5
Your Output (stdout)	
1	10 10
2	10 11

4. Given a circular integer array `nums` of length `n`, return the maximum possible sum of a non-empty subarray of `nums`.

A circular array means the end of the array connects to the beginning of the array. Formally, the next element of `nums[i]` is `nums[(i + 1) % n]` and the previous element of `nums[i]` is `nums[(i - 1 + n) % n]`.

A subarray may only include each element of the fixed buffer `nums` at most once.
Formally, for a subarray `nums[i]`, `nums[i + 1]`, ..., `nums[j]`, there does not exist $i \leq k_1$, $k_2 \leq j$ with $k_1 \% n == k_2 \% n$.

Example 1:

Input: `nums = [1,-2,3,-2]`

Output: 3

Explanation: Subarray `[3]` has maximum sum 3.

Example 2:

Input: `nums = [5,-3,5]`

Output: 10

Explanation: Subarray `[5,5]` has maximum sum $5 + 5 = 10$.

Example 3:

Input: `nums = [-3,-2,-3]`

Output: -2

Explanation: Subarray `[-2]` has maximum sum -2.

Constraints:

$n == \text{nums.length}$

$1 \leq n \leq 3 * 10^4$

$-3 * 10^4 \leq \text{nums}[i] \leq 3 * 10^4$

Program:

```
class Solution {  
    public int maxSubarraySumCircular(int[] nums) {  
        int totalSum = 0;  
  
        int currMax = nums[0];  
        int maxSum = nums[0];  
  
        int currMin = nums[0];  
        int minSum = nums[0];  
  
        for (int num : nums) {  
            totalSum += num;  
        }  
  
        for (int i = 1; i < nums.length; i++) {  
            currMax = Math.max(nums[i], currMax + nums[i]);  
            maxSum = Math.max(maxSum, currMax);  
  
            currMin = Math.min(nums[i], currMin + nums[i]);  
            minSum = Math.min(minSum, currMin);  
        }  
  
        // All elements are negative  
        if (maxSum < 0) {  
            return maxSum;  
        }  
    }  
}
```

```

        return Math.max(maxSum, totalSum - minSum);
    }
}

```

Output:



5. Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

Whitespace: Ignore any leading whitespace (" ").

Signedness: Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.

Conversion: Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.

Rounding: If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31} , and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

Return the integer as the final result.

Example 1:

Input: s = "42"

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader position.

Step 1: "42" (no characters read because there is no leading whitespace)

^

Step 2: "42" (no characters read because there is neither a '-' nor '+')

^

Step 3: "42" ("42" is read in)

^

Example 2:

Input: s = "-042"

Output: -42

Explanation:

Step 1: "-042" (leading whitespace is read and ignored)

^

Step 2: "-042" ('-' is read, so the result should be negative)

^

Step 3: "-042" ("042" is read in, leading zeros ignored in the result)

^

Example 3:

Input: s = "1337c0d3"

Output: 1337

Explanation:

Step 1: "1337c0d3" (no characters read because there is no leading whitespace)

^

Step 2: "1337c0d3" (no characters read because there is neither a '-' nor '+')

^

Step 3: "1337c0d3" ("1337" is read in; reading stops because the next character is a non-digit)

^

Example 4:

Input: s = "0-1"

Output: 0

Explanation:

Step 1: "0-1" (no characters read because there is no leading whitespace)

^

Step 2: "0-1" (no characters read because there is neither a '-' nor '+')

^

Step 3: "0-1" ("0" is read in; reading stops because the next character is a non-digit)

^

Example 5:

Input: s = "words and 987"

Output: 0

Explanation:

Reading stops at the first non-digit character 'w'.

Constraints:

$0 \leq s.length \leq 200$

s consists of English letters (lower-case and upper-case), digits (0-9), ' ', '+', '-', and '.'.

program:

```
class Solution {
    public int myAtoi(String s) {
        int i = 0;
        int n = s.length();

        // 1. Skip leading whitespaces
        while (i < n && s.charAt(i) == ' ') {
            i++;
        }
    }
}
```

```

// If string contains only spaces
if (i == n) {
    return 0;
}

// 2. Check sign
int sign = 1;
if (s.charAt(i) == '-') {
    sign = -1;
    i++;
} else if (s.charAt(i) == '+') {
    i++;
}

// 3. Convert digits
int result = 0;

while (i < n && Character.isDigit(s.charAt(i))) {
    int digit = s.charAt(i) - '0';

    // 4. Check overflow
    if (result > (Integer.MAX_VALUE - digit) / 10) {
        return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE;
    }

    result = result * 10 + digit;
    i++;
}

```

```

    }

    return sign * result;
}
}

```

Output:

Input (stdin)	
1	5
2	AAAA
3	BBBBB
4	ABABABAB
5	BABABA
6	AAABBB
Your Output (stdout)	
1	3
2	4
3	0

6. You are given a string containing characters `a` and `b` only. Your task is to change it into a string such that there are no matching adjacent characters. To do this, you are allowed to delete zero or more characters in the string.

Your task is to find the minimum number of required deletions.

Example

Remove an `a` at positions `2` and `3` to make `abab` deletions.

Function Description

Complete the `alternatingCharacters` function in the editor below.

`alternatingCharacters` has the following parameter(s):

string s: a string

Returns

int: the minimum number of deletions required

Input Format

The first line contains an integer , the number of queries.

The next lines each contain a string to analyze.

Constraints

Each string will consist only of characters and .

Sample Input

5

AAAA

BBBBB

ABABABAB

BABABA

AAABBB

Sample Output

3

4

0

0

4

Explanation

The characters marked red are the ones that can be deleted so that the string does not have matching adjacent characters.image

Program:

```
import java.io.*;

import java.math.*;

import java.security.*;

import java.text.*;

import java.util.*;

import java.util.concurrent.*;

import java.util.function.*;

import java.util.regex.*;

import java.util.stream.*;

import static java.util.stream.Collectors.joining;

import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'alternatingCharacters' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts STRING s as parameter.
     */

    public static int alternatingCharacters(String s) {

        // Write your code here
    }
}
```

```

int deletions = 0;

for (int i = 1; i < s.length(); i++) {
    if (s.charAt(i) == s.charAt(i - 1)) {
        deletions++;
    }
}

return deletions;

}

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
        InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        int q = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, q).forEach(qItr -> {
            try {
                String s = bufferedReader.readLine();

                int result = Result.alternatingCharacters(s);

```

```

        bufferedWriter.write(String.valueOf(result));

        bufferedWriter.newLine();
    } catch (IOException ex) {
        throw new RuntimeException(ex);
    }
});

bufferedReader.close();
bufferedWriter.close();
}
}

```

Output:

Input (stdin)	
1	5
2	AAAA
3	BBBBB
4	ABABABAB
5	BABABA
6	AAABBB
Your Output (stdout)	
1	3
2	4
3	6

7. Given a string *s*, find the length of the longest substring without duplicate characters.

Example 1:

Input: *s* = "abcabcbb"

Output: 3

Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: s = "bbbbbb"

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: s = "pwwkew"

Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:

$0 \leq s.length \leq 105$

s consists of English letters, digits, symbols and spaces.

Program:

```
class Solution {  
    public int lengthOfLongestSubstring(String s) {  
        Set<Character> set = new HashSet<>();  
  
        int left = 0;  
        int maxLength = 0;  
  
        for (int right = 0; right < s.length(); right++) {
```

```

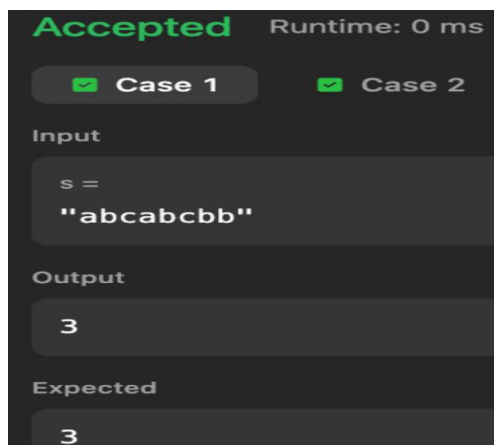
while (set.contains(s.charAt(right))) {
    set.remove(s.charAt(left));
    left++;
}

set.add(s.charAt(right));
maxLength = Math.max(maxLength, right - left + 1);
}

return maxLength;
}
}

```

Output:



8.

Given a list of strings words and a string pattern, return a list of words[i] that match pattern. You may return the answer in any order.

A word matches the pattern if there exists a permutation of letters p so that after replacing every letter x in the pattern with p(x), we get the desired word.

Recall that a permutation of letters is a bijection from letters to letters: every letter maps to another letter, and no two letters map to the same letter.

Example 1:

Input: words = ["abc","deq","mee","aqq","dkd","ccc"], pattern = "abb"

Output: ["mee","aqq"]

Explanation: "mee" matches the pattern because there is a permutation {a -> m, b -> e, ...}.

"ccc" does not match the pattern because {a -> c, b -> c, ...} is not a permutation, since a and b map to the same letter.

Example 2:

Input: words = ["a","b","c"], pattern = "a"

Output: ["a","b","c"]

Constraints:

1 <= pattern.length <= 20

1 <= words.length <= 50

words[i].length == pattern.length

pattern and words[i] are lowercase English letters.

Program:

```
class Solution {  
    public List<String> findAndReplacePattern(String[] words, String pattern) {  
        List<String> result = new ArrayList<>();
```

```
for (String word : words) {  
    if (matches(word, pattern)) {  
        result.add(word);  
    }  
}  
  
return result;  
}
```

```
private boolean matches(String word, String pattern) {  
    Map<Character, Character> pToW = new HashMap<>();  
    Map<Character, Character> wToP = new HashMap<>();  
  
    for (int i = 0; i < word.length(); i++) {  
        char p = pattern.charAt(i);  
        char w = word.charAt(i);  
  
        if (pToW.containsKey(p)) {  
            if (pToW.get(p) != w) {  
                return false;  
            }  
        } else {  
            pToW.put(p, w);  
        }  
  
        if (wToP.containsKey(w)) {  
            if (wToP.get(w) != p) {  
                return false;  
            }  
        }  
    }  
}
```

```

        }
    } else {
        wToP.put(w, p);
    }
}

return true;
}
}

```

Output:

```

words =
["abc", "deq", "mee", "aqq", "dkd", "ccc"]

pattern =
"abb"

Output
["mee", "aqq"]

```

9. Given an array of string words, return all strings in words that are a substring of another word. You can return the answer in any order.

Example 1:

Input: words = ["mass","as","hero","superhero"]

Output: ["as","hero"]

Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".

["hero","as"] is also a valid answer.

Example 2:

Input: words = ["leetcode","et","code"]

Output: ["et","code"]

Explanation: "et", "code" are substring of "leetcode".

Example 3:

Input: words = ["blue","green","bu"]

Output: []

Explanation: No string of words is substring of another string.

Constraints:

$1 \leq \text{words.length} \leq 100$

$1 \leq \text{words}[i].\text{length} \leq 30$

words[i] contains only lowercase English letters.

All the strings of words are unique.

Program:

```
import java.util.*;
```

```
class Solution {  
    public List<String> stringMatching(String[] words) {  
        List<String> ans = new ArrayList<>();  
  
        for (int i = 0; i < words.length; i++) {  
            for (int j = 0; j < words.length; j++) {
```

```

        if (i != j && words[j].contains(words[i])) {
            ans.add(words[i]);
            break;
        }
    }
}

return ans;
}
}

```

Output:

```

Input
words =
["mass","as","hero","superhero"]

Output
["as","hero"]

Expected
["as","hero"]

```

10. Given a text string of length n and a pattern of length m , find all starting indices where the pattern occurs in the text.

Note: You may assume that $n > m$.

Examples:

Input: text = "geeksforgeeks", pattern = "geeks"

Output: [0, 8]

Explanation: The string "geeks" occurs at index 0 and 8 in text.

Input: text = "aabaacaadaabaaba", pattern = "aaba"

Output: [0, 9, 12]

kmp-algorithm-for-pattern-searching

Naive Pattern Searching Algorithm

The pattern moves over the text one position at a time and characters are compared. If all characters match, the index is stored; otherwise, the next position is checked.

Program:

```
import java.util.ArrayList;
```

```
public class GFG {
```

```
    // Function to search for all occurrences of 'pat' in 'txt' using Naive Approach
```

```
    static ArrayList<Integer> search(String pat, String txt)
```

```
    {
```

```
        // Length of the pattern
```

```
        int m = pat.length();
```

```
        // Length of the text
```

```
        int n = txt.length();
```

```
        ArrayList<Integer> ans = new ArrayList<>();
```

```
        // Slide the pattern over text one by one
```

```
        for (int i = 0; i <= n - m; i++)
```

```
        {
```

```
            int j;
```

```

        // Check for pattern match at index i
        for (j = 0; j < m; j++)
        {
            if (txt.charAt(i + j) != pat.charAt(j))
                break;
        }

        // If pattern matches, store index
        if (j == m)
            ans.add(i);
    }

    return ans;
}

public static void main(String[] args)
{
    String txt = "aabaacaadaabaaba";
    String pat = "aaba";

    ArrayList<Integer> res = search(pat, txt);

    for (int it : res)
    {
        System.out.print(it + " ");
    }
}

```

```
}
```

Output:

Output



0 9 12